JOBSHEET 07

API ROUTES PADA NEXT.JS DAN INTEGRASI FIREBASE

(FULLSTACK NEXT.JS)

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

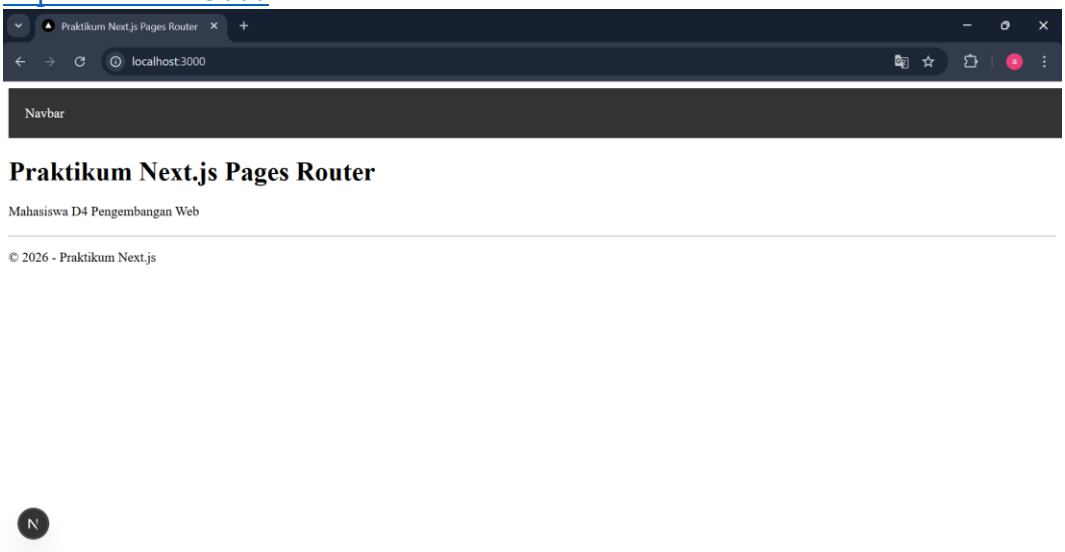
Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_07

D. Langkah Kerja Praktikum

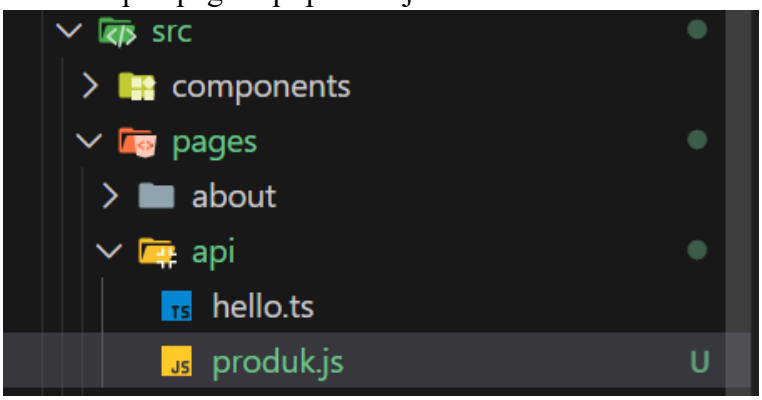
Langkah 1 – Menjalankan Project

- npm run dev
  Akses:
- http://localhost:3000



Langkah 2 – Membuat API Produk

1. Buat file pad pages/api/produk.js



2. Tambahkan data statis:

```python
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
    status: boolean;
    status_code: number;
  data: {
    id: string;
    nama: string;
    harga: number;
    ukuran: string;
    warna: string;
  }[];
};

export default function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
)
    {
    const data = [
        {
        id: "1",
        nama: "Kaos Polos",
        harga: 10000,
        ukuran: "L",
        warna: "merah",
        },
        {
        id: "2",
        nama: "Kaos Berlengan Panjang",
        harga: 15000,
        ukuran: "M",
        warna: "biru",
        },
    ];

    res.status(200).json({status:true,status_code:200,data:data });
    };
```

3. Akses: http://localhost:3000/api/produk

localhost:3000/api/produk

Pretty-print


```
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "1",
      "nama": "Kaos Polos",
      "harga": 10000,
      "ukuran": "L",
      "warna": "merah"
    },
    {
      "id": "2",
      "nama": "Kaos Berlengan Panjang",
      "harga": 15000,
      "ukuran": "M",
      "warna": "biru"
    }
  ]
}
```

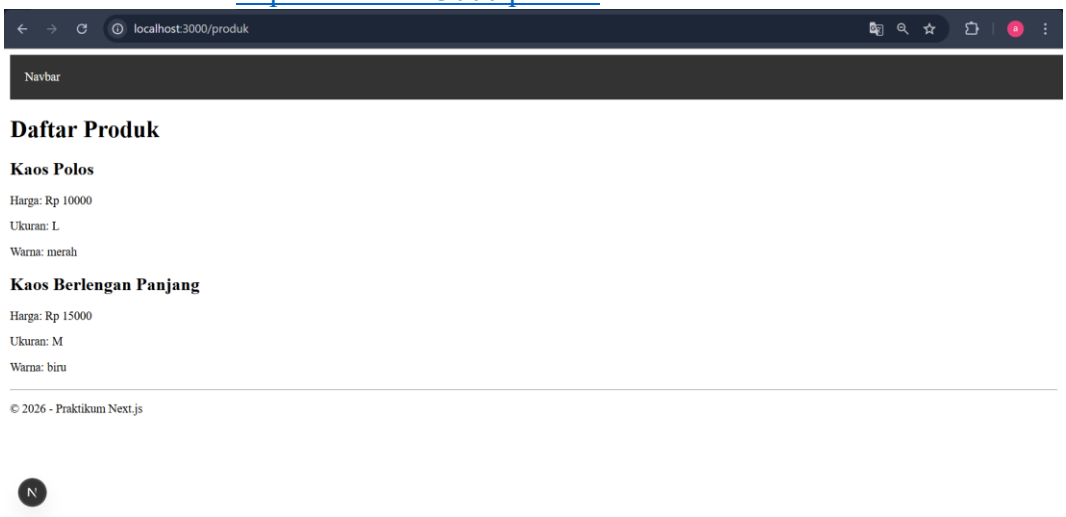
Langkah 3 – Fetch Data API di Frontend

1. Buka pages/product/index.tsx

- Modifikasi kode
  Tambahkan useEffect() dan comment useEffect untuk isLogin

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";

type ProductType = {
    id: string;
    nama: string;
    harga: number;
    ukuran: string;
    warna: string;
}

const kategori = () => {
    const [products, setProducts] = useState([]);
    // const ProdukPage = () => {
    // const { push } = useRouter();

    // Cek status login saat halaman dibuka
    // useEffect(() => {
    //     const loginStatus = localStorage.getItem("isLogin");

    //     if (loginStatus !== "true") {
    //         push("/auth/login");
    //     }
    // }, []);

    useEffect(() => {
        fetch("/api/produk")
            .then((response) => response.json())
            .then((responsdata) => {
                //console.log("Data produk:", responsdata.data);
                setProducts(responsdata.data);
            })
            .catch((error) => {
                console.error("Error fetching produk:", error);
            });
    }, []);

    return (
        <div>
        <h1>Daftar Produk</h1>
        {products.map((product: ProductType) => (
            <div key={product.id}>
                <h2>{product.nama}</h2>
                <p>Harga: Rp {product.harga}</p>
                <p>Ukuran: {product.ukuran}</p>
                <p>Warna: {product.warna}</p>
            </div>
        ))}
        </div>
    );
};

export default kategori;
```

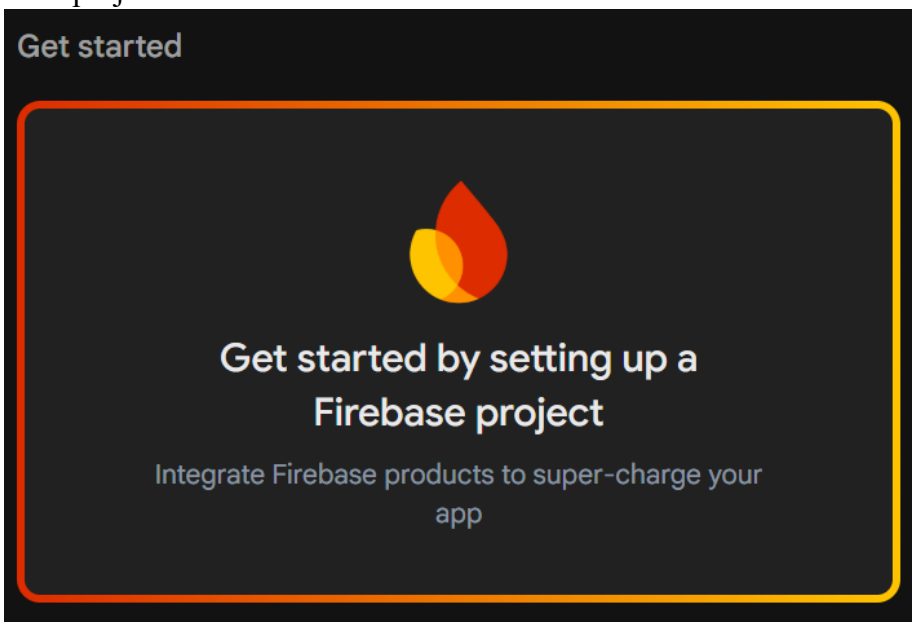
2. Jalankan browser http://localhost:3000/produk

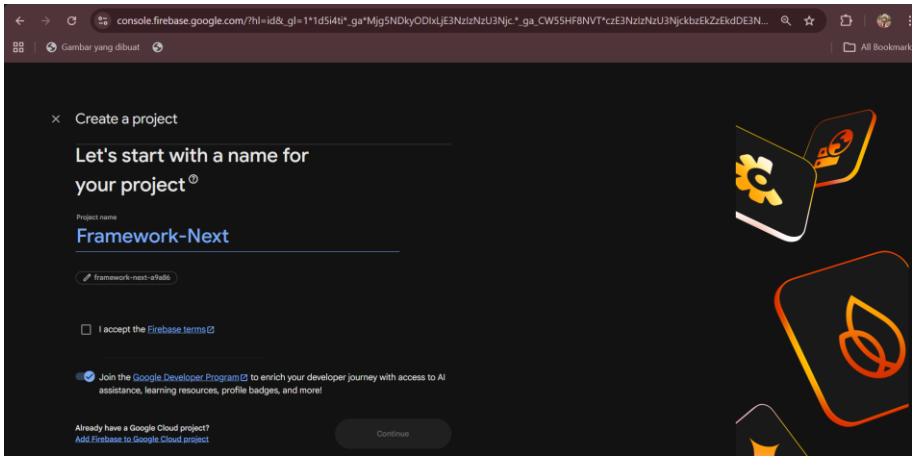


E. Integrasi Firebase

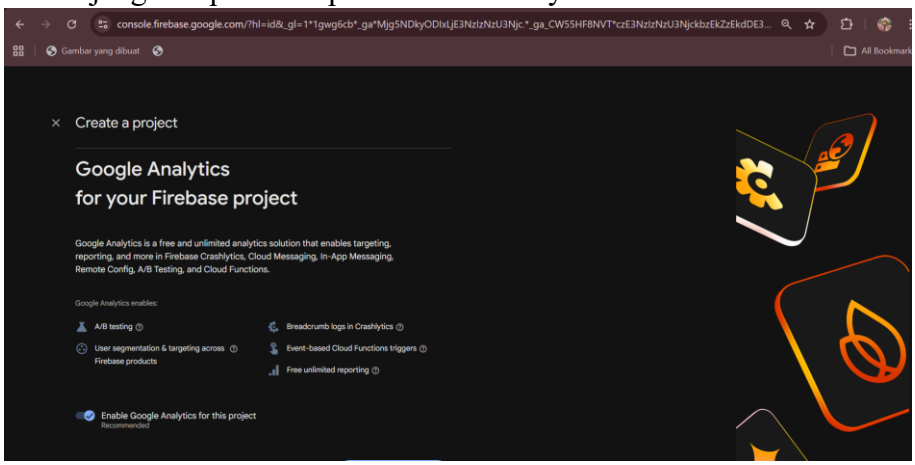
Langkah 5 – Setup Firebase

1. Buka Firebase Go To Console (login dengan login google)
2. Buat project baru

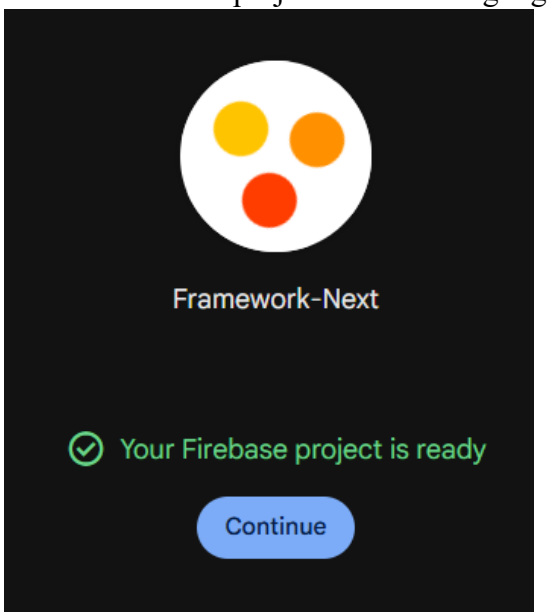


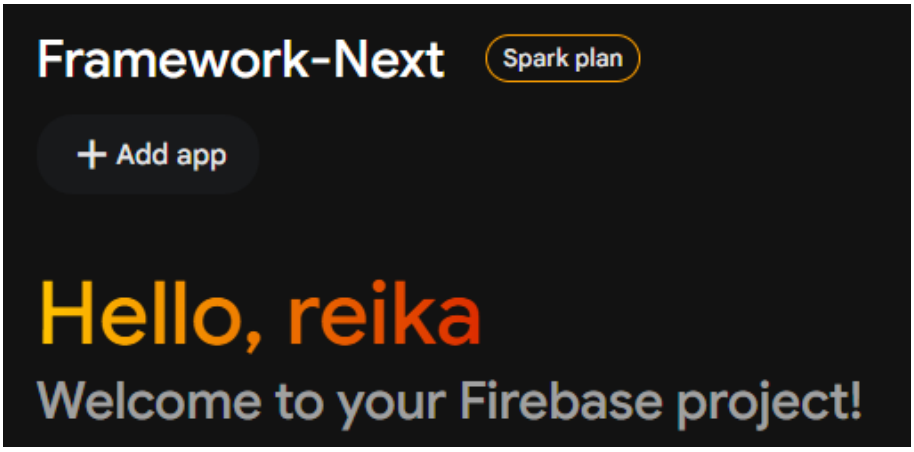


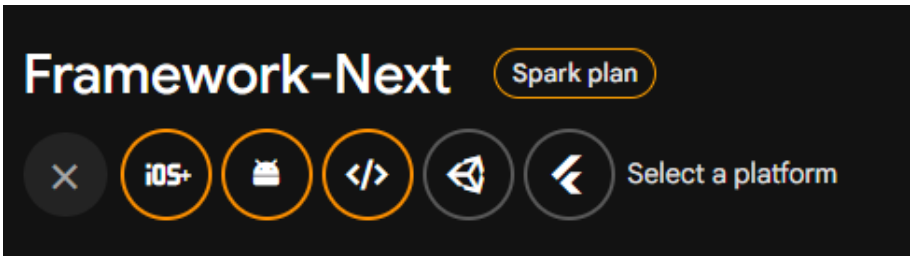
Note : jangan lupa select parent resource nya



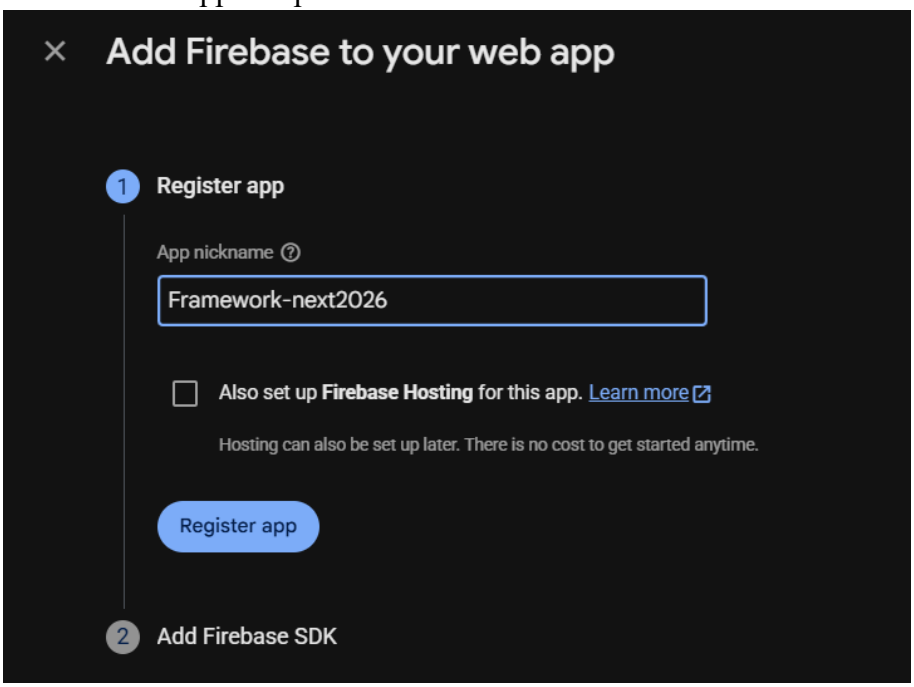
Note : klik create project dan disable google analytics



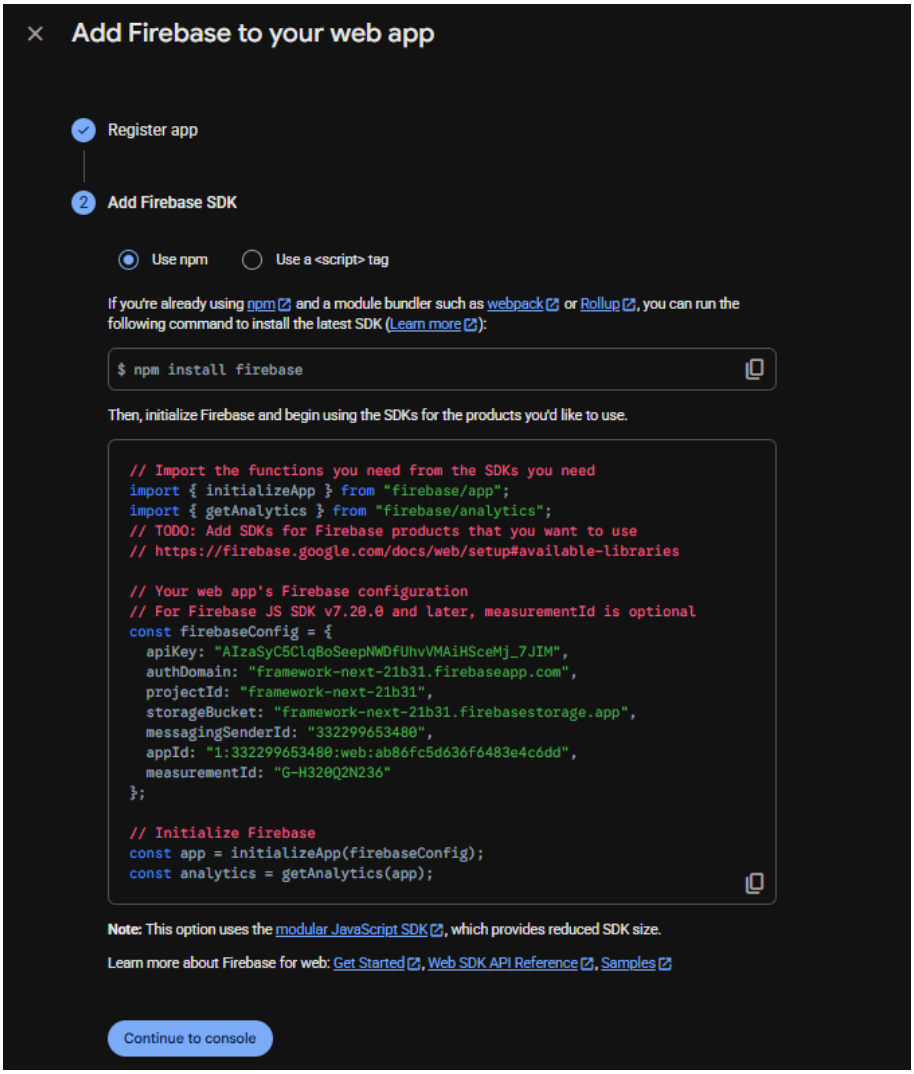




Note klik add app dan pilih web

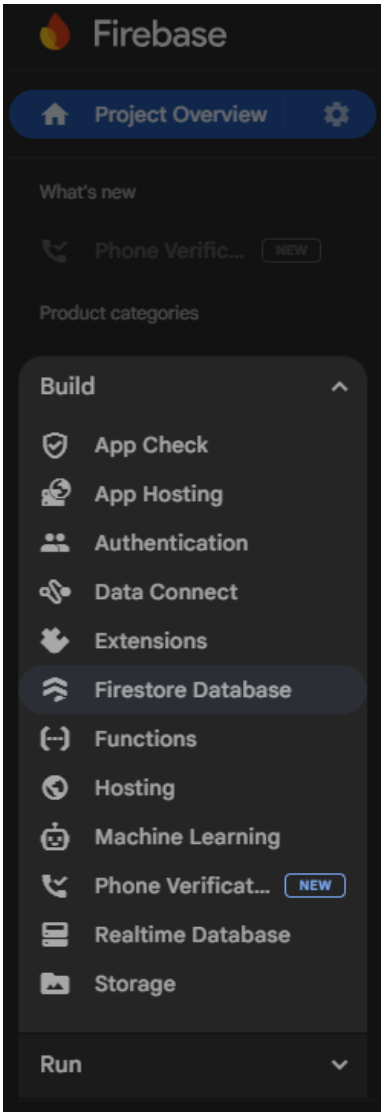


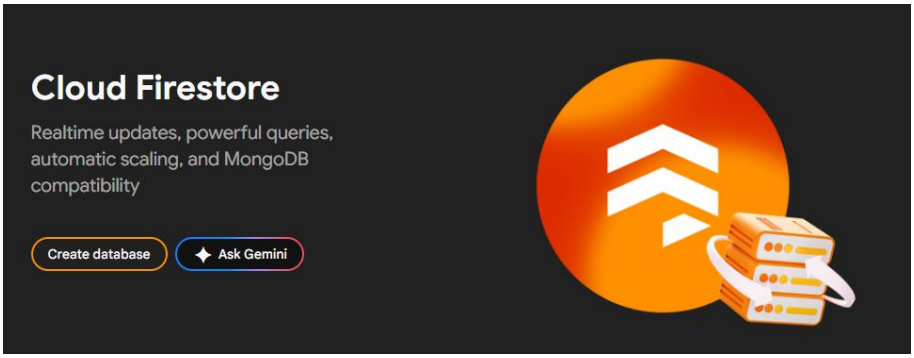
Note : klik register app



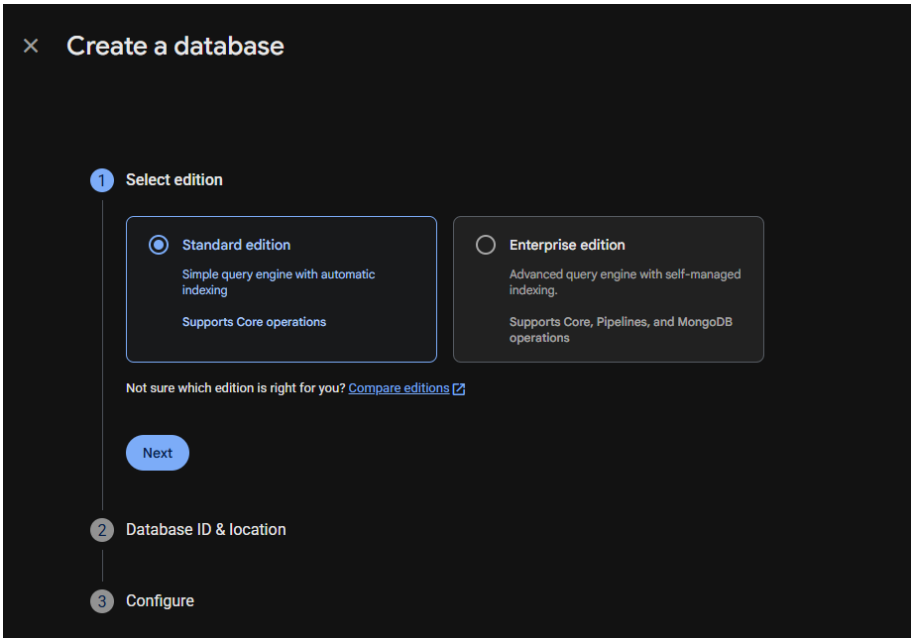
Note : klik continue to console

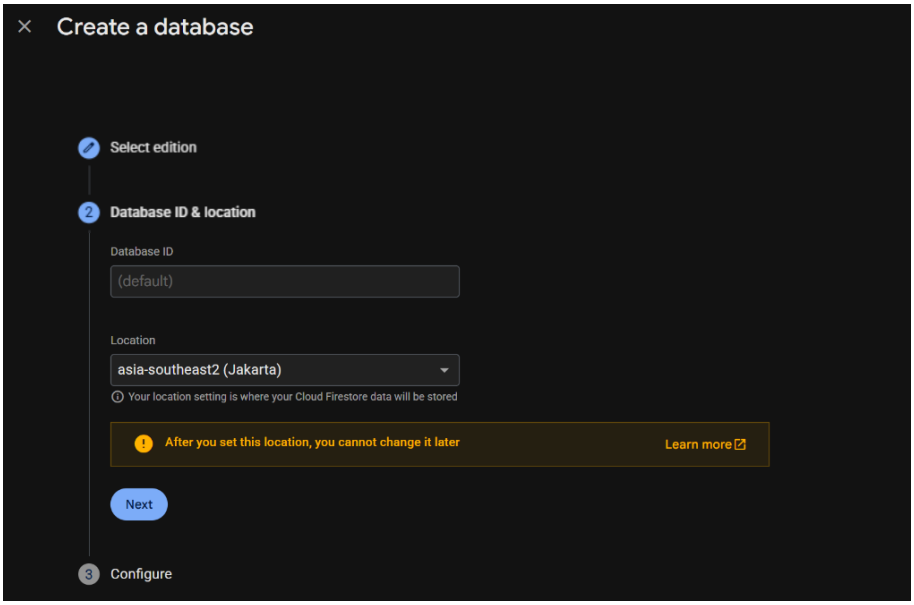
3. Aktifkan Firestore Database

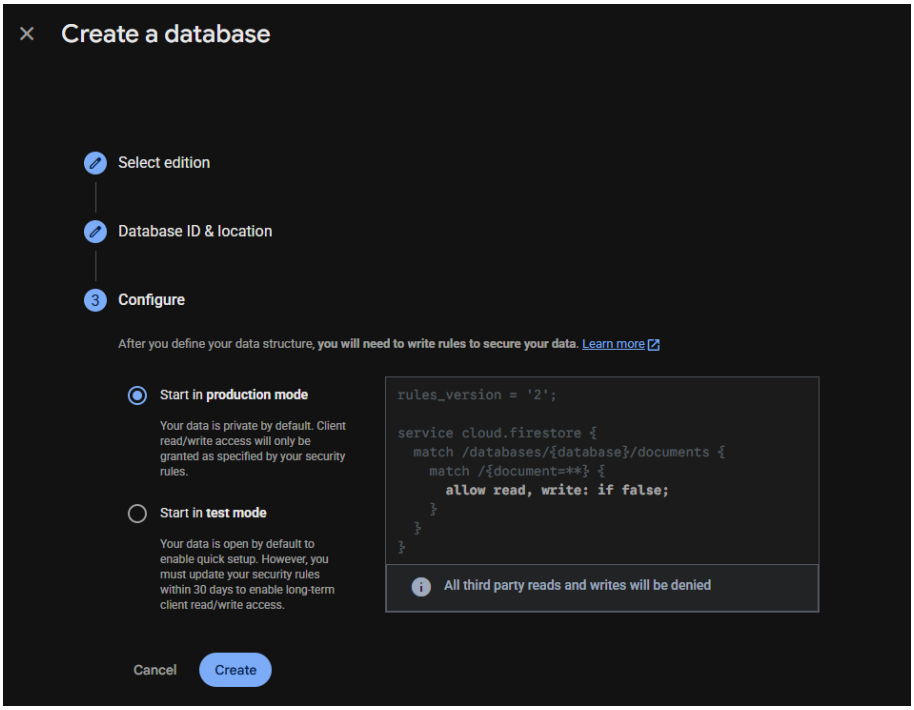


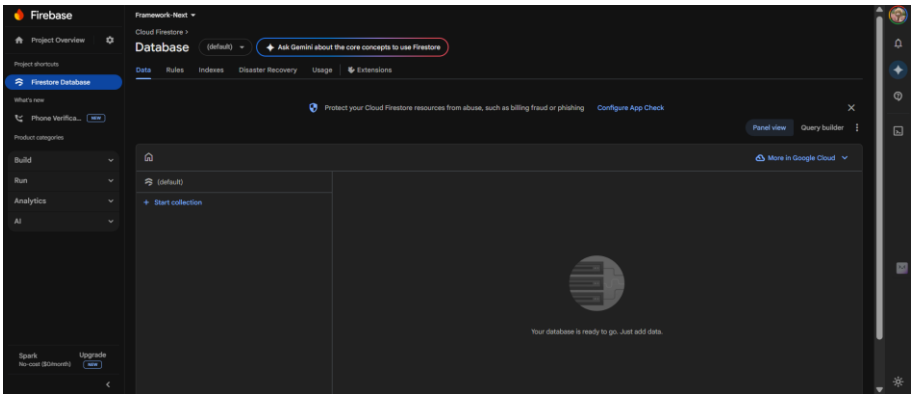


Note : klik create database

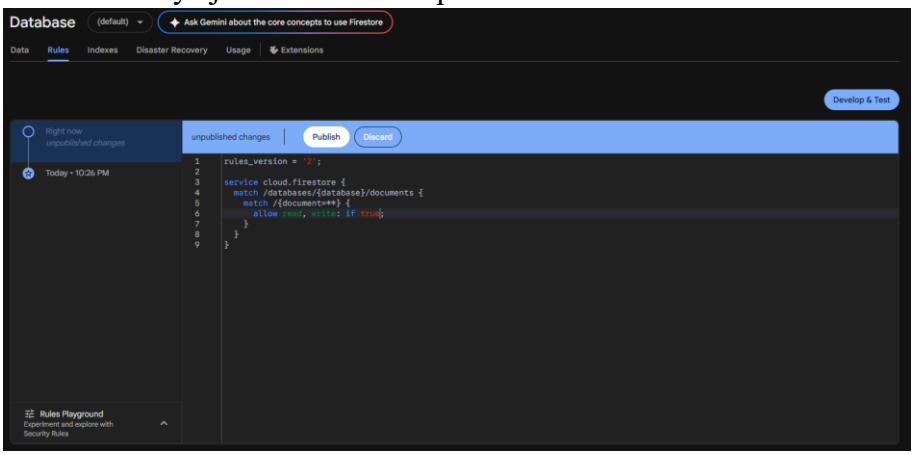

Create a database
1 Select edition
Standard edition
Simple query engine with automatic indexing
Supports Core operations
Enterprise edition
Advanced query engine with self-managed indexing.
Supports Core, Pipelines, and MongoDB operations
Not sure which edition is right for you? Compare editions
Next
2 Database ID & location
3 Configure


Create a database
Select edition
2 Database ID & location
Database ID
(default)
Location
asia-southeast2 (Jakarta)
Your location setting is where your Cloud Firestore data will be stored
After you set this location, you cannot change it later
Learn more
Next
3 Configure

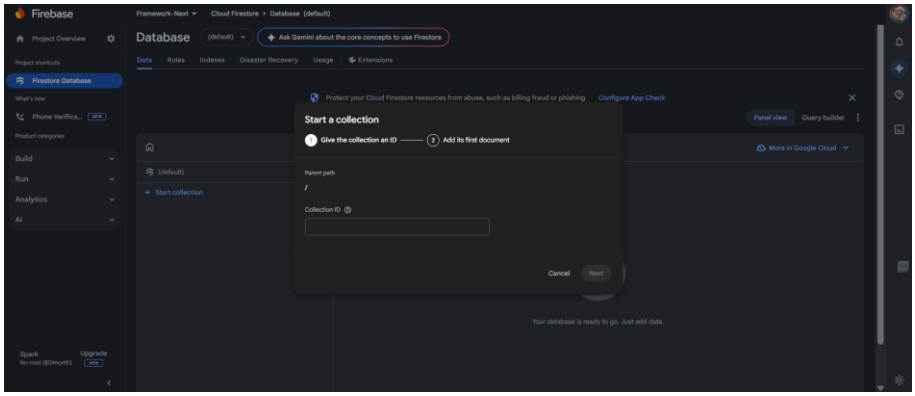




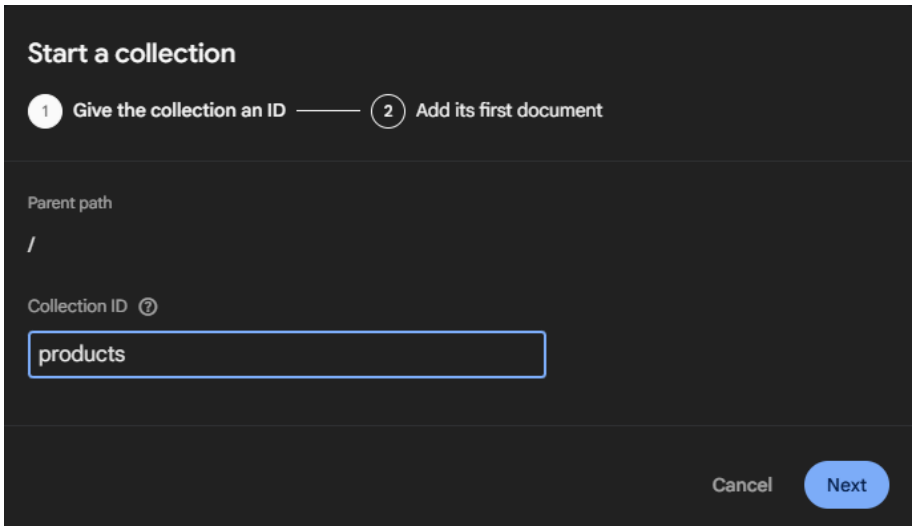
Rubah rulesnya jadi true dan klik publish



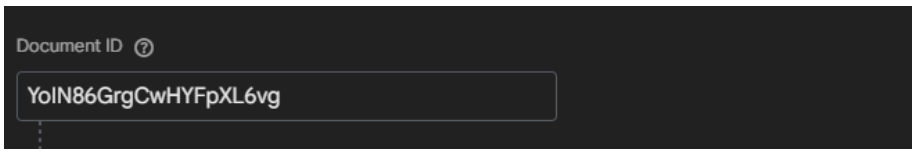
4. Buat collection:



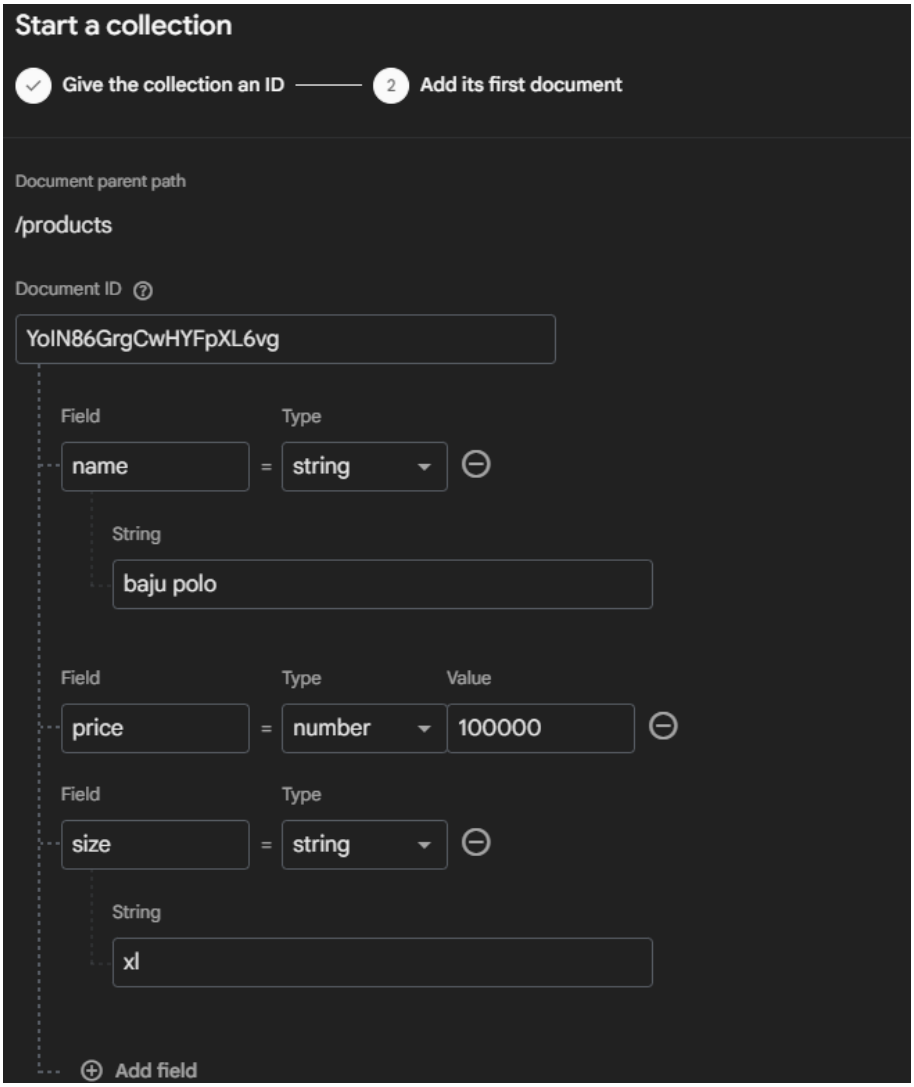
5. Products

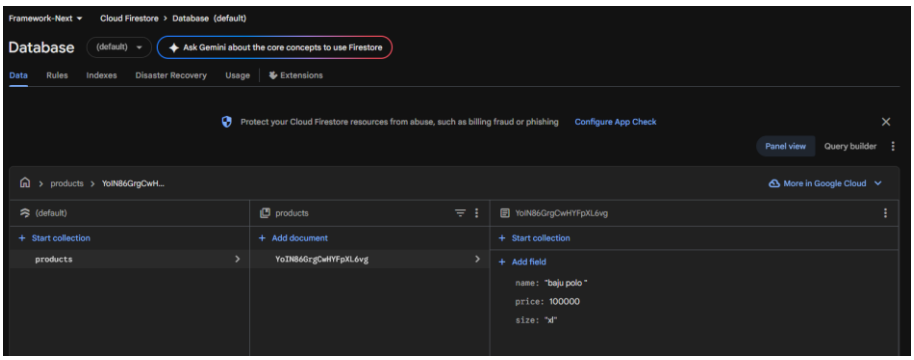


Gunakan auto-id



6. Tambahkan field:



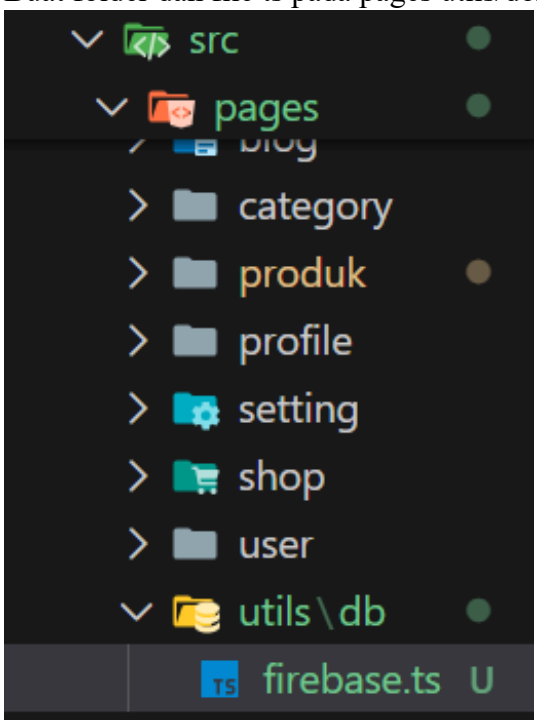


Langkah 6 – Install Firebase

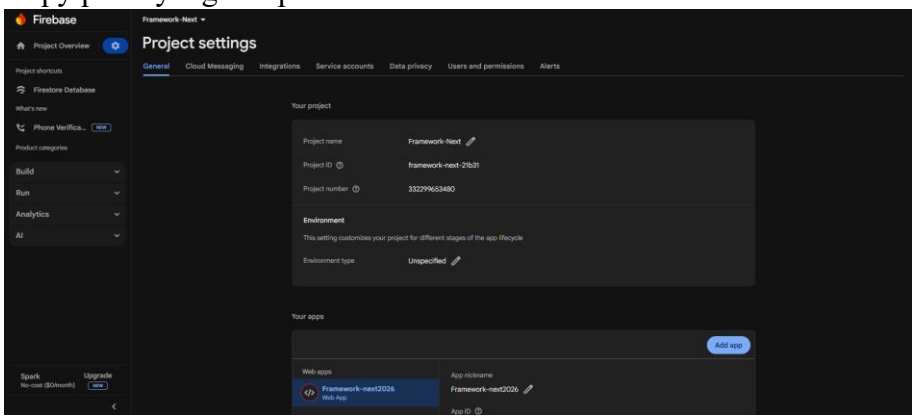
1. npm install firebase

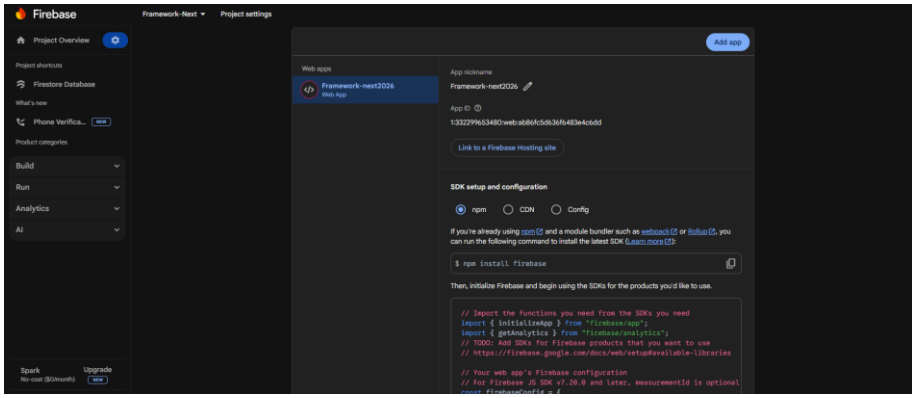
```
PS D:\Pemograman_Framework\praktikum_07\my-app-07> npm install firebase
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'eslint-visitor-keys@5.0.1',
npm WARN EBADENGINE   required: { node: '^20.19.0 || ^22.13.0 || >=24' },
npm WARN EBADENGINE   current: { node: 'v20.11.1', npm: '10.2.4' }
npm WARN EBADENGINE }

added 79 packages, and audited 429 packages in 51s

144 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

2. Buat folder dan file ts pada pages utlis/db/firebase.ts



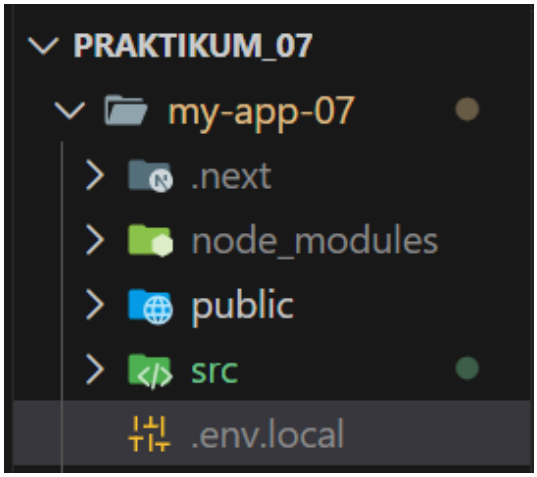
3. Copy paste yang ada pada kotak merah ke file firebase.ts



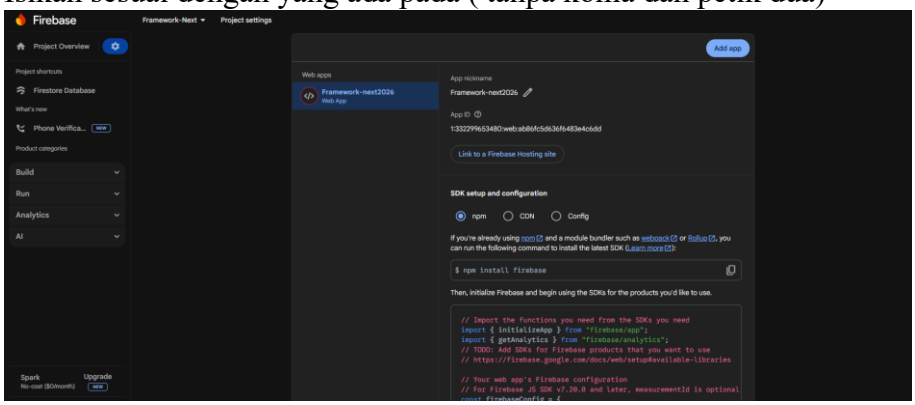


Langkah 7 – Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

1. Buat file: .env.local



2. Modifikasi file env
   FIREBASE_API_KEY=xxxx, FIREBASE_AUTH_DOMAIN=xxxx, FIREBASE_PROJECT_ID=xxxx, FIREBASE_STORAGE_BUCKET=xxxx, FIREBASE_MESSAGING_SENDER_ID=xxxx, FIREBASE_APP_ID=xxxx
   Isikan sesuai dengan yang ada pada (tanpa koma dan petik dua)



```
.env.local ×   firebase.ts U
my-app-07 > .env.local
1   FIREBASE_API_KEY=AIz
2   FIREBASE_AUTH_DOMAIN
3   FIREBASE_PROJECT_ID=
4   FIREBASE_STORAGE_BUC
5   FIREBASE_MESSAGING_S
6   FIREBASE_APP_ID=1:33
```

Langkah 8 – Konfigurasi Firebase

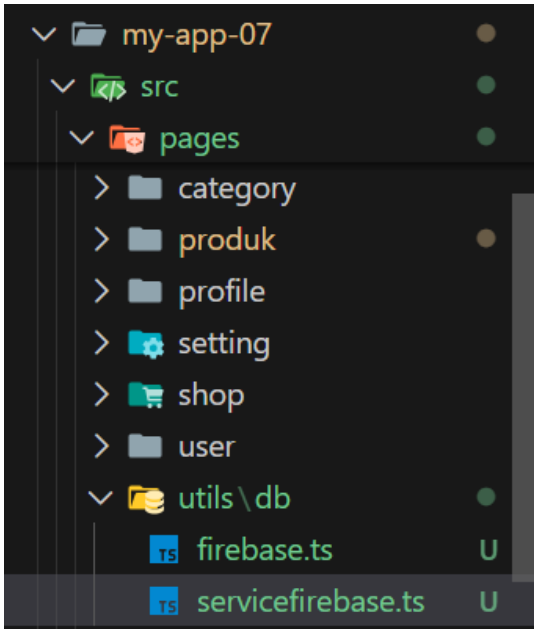
1. Modifikasi firebase.ts

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: process.env.FIREBASE_API_KEY,
  authDomain: process.env.FIREBASE_AUTH_DOMAIN,
  projectId: process.env.FIREBASE_PROJECT_ID,
  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.FIREBASE_APP_ID,
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);

export default app;
```

Langkah 9 – Ambil Data dari Firestore

1. Buat file

- ulits/db/servicefirebase.js



- modifikasi file servicefirebase.js

```
import { getFirestore, collection, getDocs, doc } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
    const snapshot = await getDocs(collection(db, collectionName));
    const data = snapshot.docs.map((doc) => ({
    id: doc.id,
        ...doc.data(),
    }));
    return data;
}
```

Langkah 10 – API Mengambil Data Firebase

1. Edit pages/api/product.ts:

```python
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";
import { retrieveProducts } from "../utils/db/servicefirebase";

type Data = {
    status: boolean;
    status_code: number;
    data: any;
  };

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
) {
    const data = await retrieveProducts("products");
    res.status(200).json({
        status: true,
        status_code: 200,
        data
    });
};
```

2. Jalankan browser http://localhost:3000/api/produk

localhost:3000/api/produk

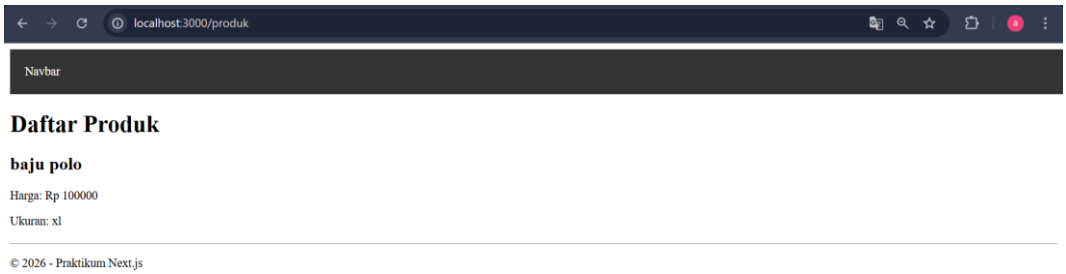
Pretty-print


```
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "YoIN8GGrgCwHYFpXL6vg",
      "size": "xl",
      "price": 100000,
      "name": "baju polo "
    }
  ]
}
```


3. Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

```python
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";

type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
}

const kategori = () => {
    const [products, setProducts] = useState([]);
    // const ProdukPage = () => {
    // const { push } = useRouter();

    // Cek status login saat halaman dibuka
    // useEffect(() => {
    //     const loginStatus = localStorage.getItem("isLogin");

    //     if (loginStatus !== "true") {
    //         push("/auth/login");
    //     }
    // }, []);

    useEffect(() => {
        fetch("/api/produk")
            .then((response) => response.json())
            .then((responsdata) => {
                //console.log("Data produk:", responsdata.data);
                setProducts(responsdata.data);
            })
            .catch((error) => {
                console.error("Error fetching produk:", error);
            });
    }, []);

    return (
        <div>
        <h1>Daftar Produk</h1>
        {products.map((products: ProductType) => (
            <div key={products.id}>
                <h2>{products.name}</h2>
                <p>Harga: Rp {products.price}</p>
                <p>Ukuran: {products.size}</p>
            </div>
        ))}
        </div>
    );
};

export default kategori;
```

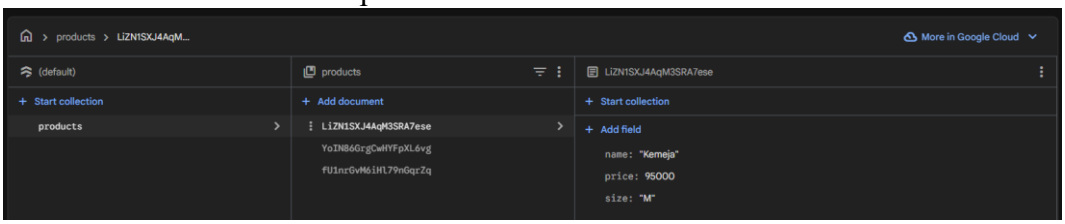
Jalankan browser



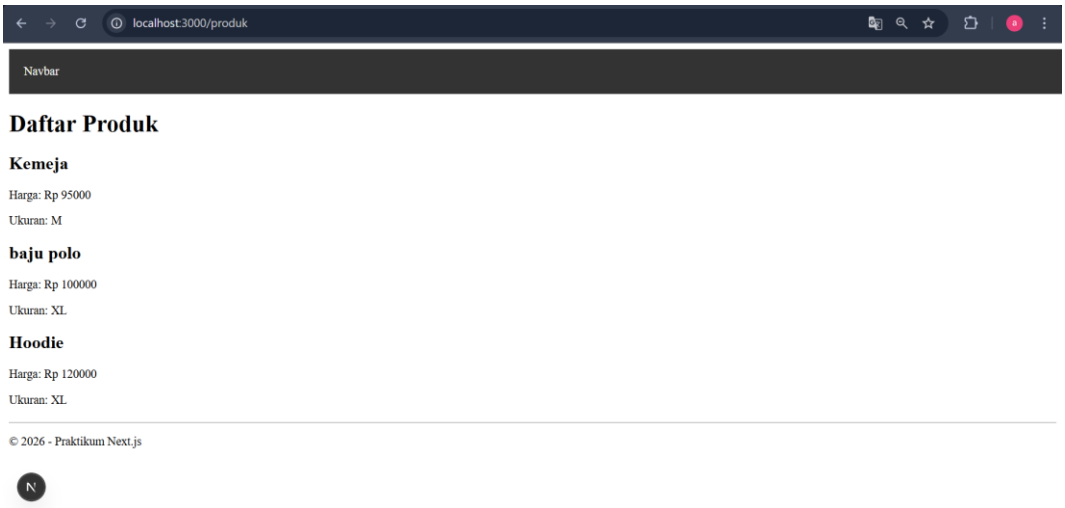
F. Tugas Praktikum

Tugas 1 (Wajib)

- Tambahkan minimal 3 data produk di Firestore



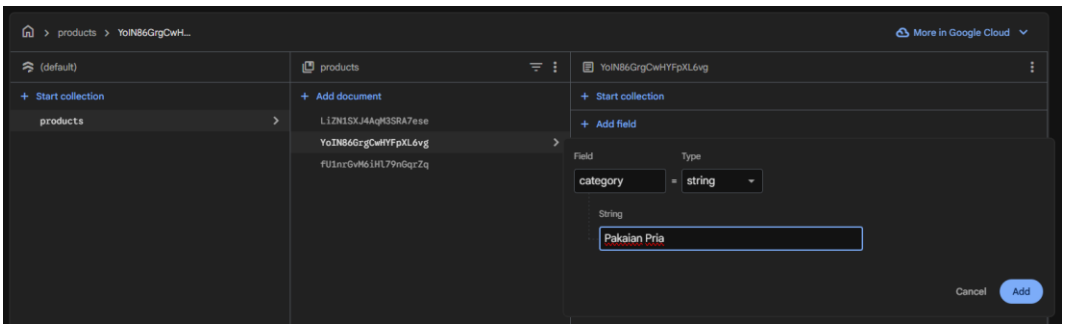
- Pastikan data tampil di halaman produk

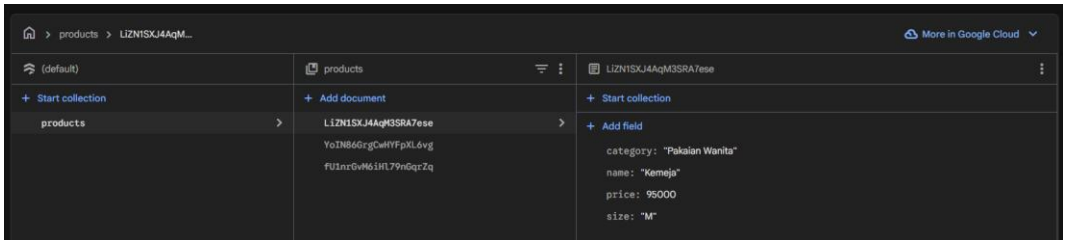


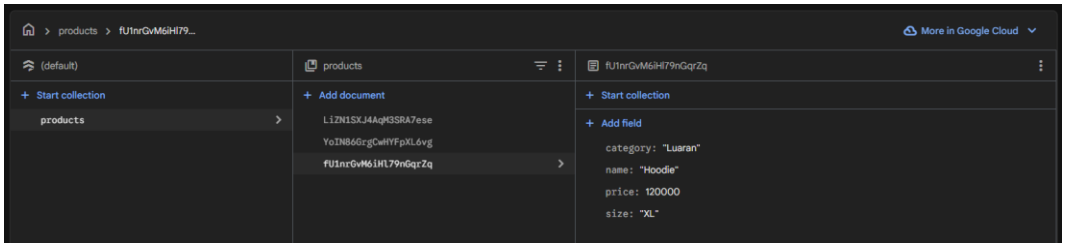
Tugas 2 (Wajib)

- Tambahkan field baru:

  category



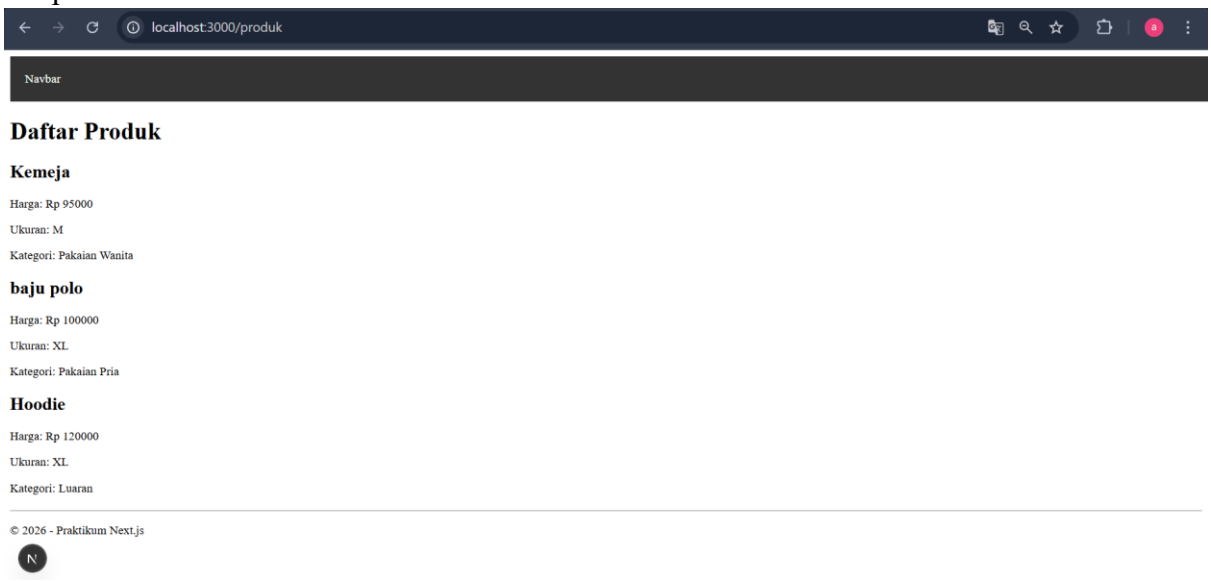




- Tampilkan category di frontend
  - Modifikasi bagian folder produk pada file index

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";

type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
    category: string;
}

const kategori = () => {
    const [products, setProducts] = useState([]);
    // const ProdukPage = () => {
    // const { push } = useRouter();

    // Cek status login saat halaman dibuka
    // useEffect(() => {
    //     const loginStatus = localStorage.getItem("isLogin");

    //     if (loginStatus !== "true") {
    //         push("/auth/login");
    //     }
    // }, []);

    useEffect(() => {
        fetch("/api/produk")
            .then((response) => response.json())
            .then((responsdata) => {
                //console.log("Data produk:", responsdata.data);
                setProducts(responsdata.data);
            })
            .catch((error) => {
                console.error("Error fetching produk:", error);
            });
    }, []);

    return (
        <div>
        <h1>Daftar Produk</h1>
        {products.map((products: ProductType) => (
            <div key={products.id}>
                <h2>{products.name}</h2>
                <p>Harga: Rp {products.price}</p>
                <p>Ukuran: {products.size}</p>
                <p>Kategori: {products.category}</p>
            </div>
        ))}
        </div>
    );
};

export default kategori;
```

Output:



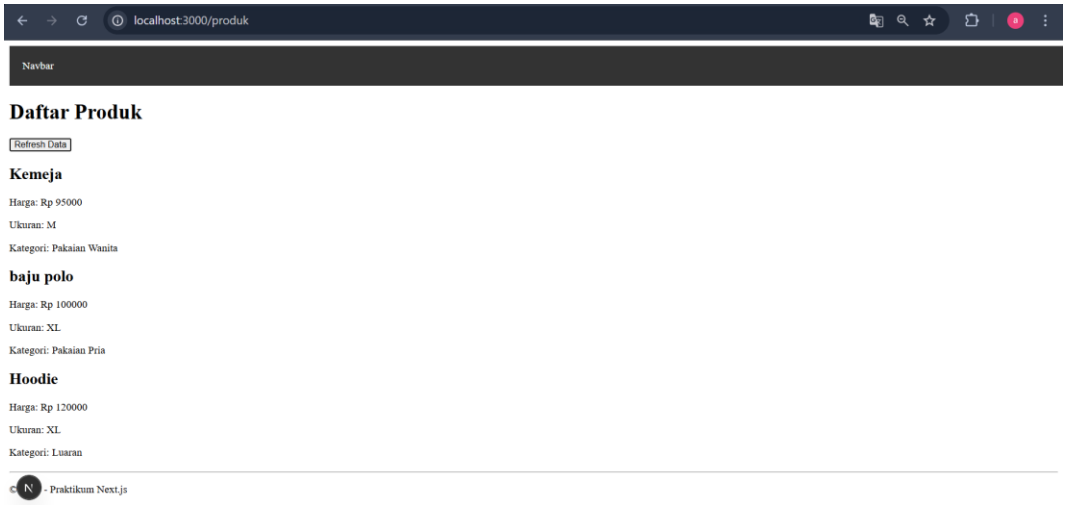
Tugas 3 (Pengayaan)

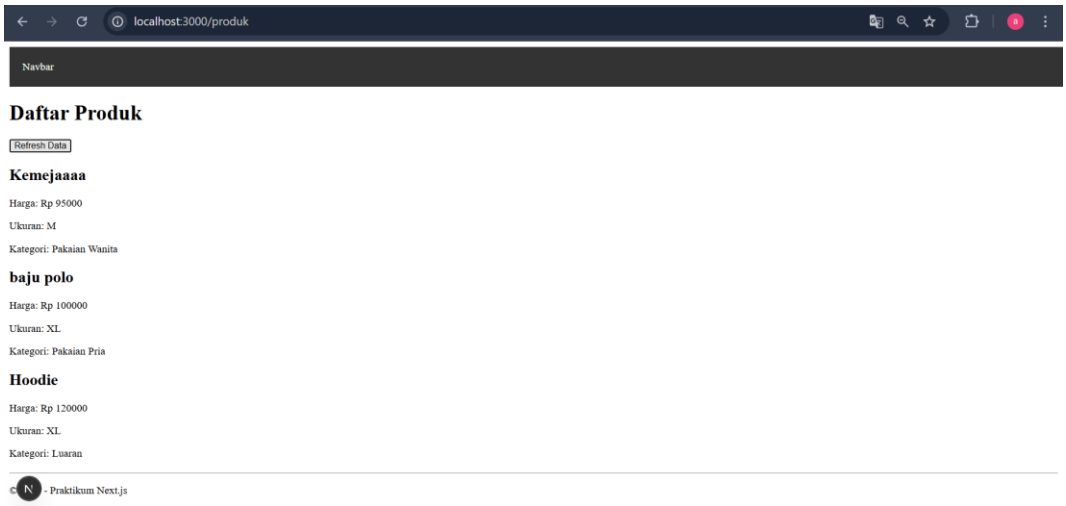
- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman
  - Saya memisahkan proses pengambilan data ke dalam sebuah fungsi bernama fetchProducts(). Fungsi ini dipanggil saat halaman pertama kali dibuka melalui useEffect, dan juga dipanggil kembali ketika tombol "Refresh Data" diklik. Dengan menggunakan setProducts(), state diperbarui sehingga React melakukan render ulang tampilan tanpa perlu me-refresh halaman secara keseluruhan.

    Modifikasi bagian folder produk pada file index

```python
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";

type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
    category: string;
}

const kategori = () => {
    const [products, setProducts] = useState([]);
    // const ProdukPage = () => {
    // const { push } = useRouter();

    // Cek status login saat halaman dibuka
    // useEffect(() => {
    //     const loginStatus = localStorage.getItem("isLogin");

    //     if (loginStatus !== "true") {
    //         push("/auth/login");
    //     }
    // }, []);

    const fetchProducts = () => {
    fetch("/api/produk")
      .then((response) => response.json())
      .then((responsdata) => {
        setProducts(responsdata.data);
      })
      .catch((error) => {
        console.error("Error fetching produk:", error);
      });
  };

  useEffect(() => {
    fetchProducts();
  }, []);

    return (
        <div>
        <h1>Daftar Produk</h1>

        <button onClick={fetchProducts}>
        Refresh Data
        </button>

        {products.map((products: ProductType) => (
            <div key={products.id}>
                <h2>{products.name}</h2>
                <p>Harga: Rp {products.price}</p>
                <p>Ukuran: {products.size}</p>
                <p>Kategori: {products.category}</p>
            </div>
        ))}
        </div>
    );
};

export default kategori;
```

- Saya mencoba menguji button Refresh Data dengan mengubah data yang ada pada firebase





G. Pertanyaan Evaluasi

1. Apa fungsi API Routes pada Next.js?
   - API Routes pada Next.js berfungsi untuk membuat backend langsung di dalam project yang sama dengan frontend. Dengan adanya API Routes, kita bisa membuat endpoint seperti /api/produk untuk mengambil, menambah, atau mengelola data.
     API Routes biasanya digunakan untuk menghubungkan aplikasi dengan database, seperti Firebase. Jadi, frontend tidak langsung mengambil data dari database, tetapi melalui API terlebih dahulu. Hal ini membuat sistem lebih terstruktur dan aman.

2. Mengapa .env.local tidak boleh di-push ke repository?
   - File .env.local berisi data konfigurasi penting seperti API key dan informasi project. Data tersebut bersifat sensitif dan tidak boleh diketahui oleh orang lain. Jika file ini di-push ke repository (GitHub), maka orang lain bisa melihat dan menyalahgunakan konfigurasi tersebut. Oleh karena itu, file .env.local biasanya dimasukkan ke dalam .gitignore agar tidak ikut terupload ke repository.

3. Apa perbedaan data statis dan data dinamis?
   - Data statis adalah data yang ditulis langsung di dalam kode program dan tidak berubah kecuali kita mengubahnya secara manual. Contohnya seperti array produk yang ditulis langsung di file JavaScript.
     Sedangkan data dinamis adalah data yang diambil dari database atau API, dan dapat berubah tanpa perlu mengubah kode program. Misalnya data produk yang disimpan di Firestore. Jika kita menambahkan produk baru di database, maka data tersebut bisa langsung tampil tanpa mengubah kode.

4. Mengapa Next.js disebut framework fullstack?
   - Next.js disebut framework fullstack karena dapat digunakan untuk membuat frontend dan backend dalam satu project yang sama.
     Frontend digunakan untuk membuat tampilan halaman, sedangkan backend dibuat melalui API Routes yang ada di folder pages/api. Dengan demikian, kita tidak perlu membuat server terpisah untuk backend karena sudah bisa dibuat langsung di dalam Next.js.